

Chapter 3. Container Images





Outline

- Navigate container registries.
- Pull and manage container images.



Container Image Registries

- **A container image** is a packaged version of your application, with all the dependencies necessary for the application to run.
- **An image registries** is used to store container images to later share them in a controlled manner. Some examples of image registries include **Quay.io, Red Hat Registry, Docker Hub, or Amazon ECR**.
- **podman pull <registry-name/repository-name/image-name>:<tag>** used to pull image from a registry.
- EX: `podman pull registry.redhat.io/ubi8/ubi:8.6`



Red Hat Registry

- Red Hat distributes container images by using two registries:
 - 1) registry.access.redhat.com (where **no** authentication is required)
 - 2) registry.redhat.io (where authentication is required)
- you have to navigate to <https://catalog.redhat.com/software/containers/explore> to specifically search for container images

Quay.io

- Although the Red Hat Registry only stores images from Red Hat and certified providers.
- you can use Quay.io, another image registry, **to store your own images**. Storing public images in Quay is free, but there are some options only available for paying customers.
- At the same time, Quay offers an on-premise version of the product, which you can use to set up an image registry in your own servers. To log in to Quay, you can use your Red Hat developer account.



The Containerfile

- A **Containerfile**, often called **Dockerfile**, is a file that contains instructions indicating **how the image is created**.
- Usually, container images are **created based on other container images**.
- For example, an image that outputs Hello world might be created by using a Linux parent image. The Linux image provides the capabilities to output text, and the child image uses the capabilities of the parent image to print the text.

```
FROM registry.redhat.io/ubi9 1  
CMD echo "Hello world" 2
```

- **FROM** instruction: indicates the parent image to use. In this case, a Universal Base Image (UBI), Version 9 container image is used.
- **CMD** instruction: executes a command. In this case, the echo command is used to output some text.



Manage Registries with Podman

- Usually, when you pull an image by using Podman, you provide the full address, that is the **registry URL, user or organization, repository, and tag**.
- However, it is also possible to specify a shorter, unqualified name, which avoids the registry URL.
- **For example**, you might shorten the `registry.redhat.io/rhel7/rhel:7.9` to **`rhel7/rhel:7.9`**.
- If you **do not provide the registry URL**, Podman does not know which registry to use, so it relies on the **`/etc/containers/registries.conf`** file of your system. This file contains the registries that Podman should use to find the image, in order of preference.

```
unqualified-search-registries = ['registry.redhat.io', 'docker.io']
```

- For example, with the following configuration, **Podman searches first in the Red Hat Registry**. If the **image is not found in the Red Hat Registry**, then Podman **searches in the Docker Hub**.
- By using the **`registries.conf` file**, you can also **block pulling from a registry**.

```
[[registry]]  
location="docker.io"  
blocked=true
```



Manage Registry Credentials with Podman

- you must execute the `podman-login <registry>` command for the registry if the registry require authentication then you can pull the image.
- Podman stores the credentials in the **`${XDG_RUNTIME_DIR}/containers/auth.json` file**, where the `${XDG_RUNTIME_DIR}` refers to a directory specific to the current user. The credentials are encoded in the base64 format.
- You can decode your password using this command :
`echo -n <password> | base64 -d`
- Old credentials maybe cached in **`~/.docker/config.json`**



Manage Registries with Skopeo

- Skopeo is a command-line tool for working with container images. Developers can use Skopeo in a number of ways, for example:

1) Inspect remote container images.

```
skopeo inspect docker://registry.access.redhat.com/ubi9/nodejs-18
```

2) Copy a container image between registries.

```
skopeo copy docker://registry.access.redhat.com/ubi9/nodejs-18  
docker://quay.io/myuser/nodejs-18
```

3) Convert image format, for example from Docker to the OCI format.

```
skopeo copy docker://registry.access.redhat.com/ubi9/nodejs-18  
dir:/var/lib/images/nodejs-18 (local dir create it)
```


Section 3.3: Managing Images





Image Versioning and Tags

- Because container images package software, developers often consider images as deployment **artifacts**.
- To keep images up-to-date, developers often map the image versions to the versions of the packaged software. Keeping images up-to-date also means updating the OS libraries within the image to receive **improvements and security fixes**.
- One way to version images relative to their packaged software product is to use **semantic versioning**. Semantic version numbers form a string with the format **MAJOR.MINOR.PATCH** meaning:
 - MAJOR: backward incompatible changes → **1.0.0** → **2.0.0** → You removed or changed an API
 - MINOR: backward compatible changes → **2.1.0** → **2.2.0**: Added new endpoint or feature
 - PATCH: bug fixes → **2.2.1** → **2.2.2**: Fixed a crash or typo



Image Versioning and Tags

- **podman pull [<image repository>/<namespace>/]<image name>[:<tag>]**
- Using a tag in Podman is optional. When you **do not specify a tag** in a Podman command, Podman uses the **latest** tag by default.
- Using the latest tag is considered a **bad practice**. Because the latest tag also represents the latest version of the image, it can include backwards-incompatible changes and cause containers that use the image to break.
- To create additional tags for local images, use the podman image tag command.
- **podman image tag LOCAL_IMAGE:TAG LOCAL_IMAGE:NEW_TAG**



Pulling Images

- **podman search <image-name>** is used to search for images in all the registries present in the **unqualified-search-registries** list in your **registries.conf** file. This enables you to search different registries in one go.
 - Pulled images are stored in your system and can be used by different containers.
- 1) User-pulled images are stored in the **~/.local/share/containers** directory.
 - `Ls ~/.local/share/containers/storage/overlay-images/`
 - 2) Root-pulled images are stored in the **/var/lib/containers** directory.
 - `sudo ls /var/lib/containers/storage/overlay-images.`
- **podman image ls** or **podman images** to list all local images.



Building Images

- **podman login <REGISTRY>**
- **podman build --file Containerfile --tag quay.io/YOUR_QUAY_USER/IMAGE_NAME:TAG** is used to build image from dockerfile.
- **podman push quay.io/YOUR_QUAY_USER/IMAGE_NAME:TAG** is used to push it to Red Hat Quay.io

Inspecting Images

- **podman image inspect <image-name> --format="{{.Config.Cmd}}"** provides useful information about a locally available image in your system.
- To select a **specific part** of the output use Podman by providing the --format option.



Image Removal

- **podman image rm** or **podman rmi** **REGISTRY/NAMESPACE/IMAGE_NAME:TAG** is used to delete local images that are no longer used by any container.
- **podman image rm -f** **REGISTRY/NAMESPACE/IMAGE_NAME:TAG**
- If the image is in use by a container, then Podman **refuses to remove it**.
 - You must first remove any containers using the image by running **podman stop container-name**.
 - Alternatively, force Podman to remove the image by providing the **-f** option. This automatically stops and removes any containers using the image and then removes the image.
- **podman rmi --all** or **podman image rm --all** is used to delete all images in the local storage.
- **podman image prune** : is used to delete all of dangling images from your local storage. (Images without tags or repository and that are not referenced by other images are considered dangling images).
- **podman image prune --all** is used to delete both dangling and unused images, provide the **--all** or **-a** option.



Export and Import Containers

- The podman **export** command exports a container's filesystem to a .tar file on your local machine. This command creates a snapshot of an existing container, so you can use it later.
 - `podman export -o mytarfile.tar <container-id>`
- The podman **import** command imports a .tar file containing a container file system, and save the file system as a container image.
- The podman import command requires the image name and tag as arguments.
 - `podman import mytarfile.tar <image-name>`

export = container → tar
import = tar → image



Save and Load Images

- The podman **save** command, unlike podman export, keeps the original image layers and metadata, such as the image history or labels.
- This command accepts the source container image name and the --output option to specify the name of the output tar file.
 - `podman save --output httpd-image.tar registry.access.redhat.com/ubi8/httpd-24`
- You can restore the saved image with the podman **load** command.
- It reads from either standard input or the file specified with the --input option.
- The podman load command does not need the container image name because it is stored as part of the image metadata.
 - `podman load --input httpd-image.tar`

Lab: section 3.5

